

Harness 架构相关文件

🕒 最后更新: 2026年04月01日 16时45分

整体架构

repo/	
├ AGENTS.md	# └ 🌟 通用OpenSpec规则
├ CLAUDE.md	# └ 🌟 Claude系统提示词
├ REVIEW.md	# └ 🌟 只读评审代理提示词
├ docs/	# └ 📖 项目知识库目录
│ └ architecture/	# └ └ 🔥 整体知识
│ │ └ index.md	# └ └ └ 项目架构总览
│ │ └ implicit-contracts.md	# └ └ └ 隐性业务约定/项目坑点
│ └ product/	# └ └ 🔥 产品知识
│ │ └ index.md	# └ └ └ 产品规则
│ └ standards/	# └ └ 🔥 相关规范
│ │ └ testing.md	# └ └ └ 测试规范
│ │ └ database.md	# └ └ └ 数据库与 SQL 规范
├ openspec/	# └ 📖 OpenSpec执行目录
│ └ changes/	# └ └ 变更目录
│ │ └ <changes名>	# └ └ └ 当前正在执行的change
│ │ │ └ specs/	# └ └ └ └ 该change的是怎么工作的
│ │ │ └ proposal.md	# └ └ └ └ 当前拆解的需求实现提案
│ │ │ └ design.md	# └ └ └ └ 执行的具体方案
│ │ │ └ task.md	# └ └ └ └ 拆解的执行步骤节点
│ │ └ archive/	# └ └ └ 归档文件
│ └ specs/	# └ └ └ 当前系统如何工作的
├ .claude/	# └ 📖 Claude项目级配置目录
├ settings.local.json.example	# └ 🔥 项目级权限设置
├ skills/	# └ 🔥 项目级Skills
│ └ prepare-review/	# └ └ PR前变更审计
│ │ └ SKILL.md	# └ └ └ Springboot分层架构检查
│ └ spring-architecture-review/	# └ └ └ SQL、Mapper、批量更新等检查
│ │ └ SKILL.md	# └ └ └
│ └ sql-risk-review/	# └ └ └
│ │ └ SKILL.md	# └ └ └
├ agents/	# └ 🔥 子代理
│ └ reviewer.md	# └ └ 只读评审代理
├ hooks/	# └ 🔥 Hook
└ guard_write.py	# └ 文件写入保护

		ensure_change_context.py	#			上下文变更保护
		run_checks.sh	#			编译检查
		src/				
		main/				
		java/				
		resources/				
		test/				
		java/				
		resources/				
		pom.xml				
		.gitignore				

具体文件

入口规则文件

AGENTS.MD

AGENTS.md

项目说明

本仓库采用 harness 风格工作流：

- OpenSpec 负责定义需求与变更工件
- Claude Code 在项目规则内执行
- 实现与评审分离
- Hooks、权限和 CI 负责硬约束

首先阅读

1. `docs/architecture/index.md`
2. `docs/product/index.md`
3. `docs/standards/testing.md`
4. `docs/standards/database.md`
5. `openspec/specs/`
6. `openspec/changes/<change-id>/`
7. `docs/architecture/implicit-contracts.md`

工作规则

- 没有 OpenSpec change，不允许直接开始开发
- 不允许超出 `tasks.md` 自行扩需求
- 每完成一个里程碑，都必须运行相关检查
- 修改数据库、配置、高风险业务时，必须明确说明影响范围
- 合并前必须经过 review 和 verify

受保护目录

- `src/main/resources/application\*.yaml`
- `src/main/resources/bootstrap\*.yaml`
- `src/main/resources/db/`
- `sql/`
- `deploy/`
- `infra/`
- `secrets/`

主流程命令

- 新需求: `/opsx:propose`
- 实施: `/opsx:apply`
- 校验: `/opsx:verify`
- 归档: `/opsx:archive`

CLAUDE.md

@AGENTS.md

Claude Code 项目规则

你的角色

你是一个在 harness 工作流中执行实现和评审任务的代理。

必须遵守的工作流程

1. 开始实现前，必须先阅读对应的 OpenSpec change:
 - `openspec/changes/<change-id>/proposal.md`
 - `openspec/changes/<change-id>/design.md`
 - `openspec/changes/<change-id>/tasks.md`
2. 修改代码前，先总结本次需求范围
3. 只允许实现 `tasks.md` 中明确列出的内容
4. 每完成一个里程碑，必须执行相关检查
5. 最终输出简短总结，包括：
 - 改动了哪些类/文件
 - 跑了哪些测试
 - 还存在哪些风险

Spring Boot 架构规则

- Controller 只负责参数接收、返回结果，不写核心业务逻辑
- 业务逻辑必须放在 Service / Domain 层
- Controller 不允许直接调用 Mapper / Repository
- Service 不允许直接依赖 Web 层对象
- DTO/VO 不允许直接当作持久化实体使用
- 涉及数据库查询变更时，必须关注索引、分页、条件范围和 N+1 风险
- 新增外部依赖时，必须在 design.md 中说明理由

测试规则

- 任何行为变化都必须补充至少一个相关测试
- Service 层变更优先补单元测试
- Controller/API 行为变更优先补集成测试
- Bug 修复在条件允许时必须补 regression test
- 修改 SQL / Mapper 时，必须至少说明验证方式

事务与数据规则

- 涉及写操作时，必须明确事务边界
- 不允许无条件全表更新/删除
- 批量操作必须说明范围控制条件
- 修改数据库脚本、迁移文件、核心 SQL 时必须谨慎，必要时停止并请求确认

安全规则

- 除非 OpenSpec design 明确要求，否则不允许修改：
 - `src/main/resources/application\*.yaml`
 - `src/main/resources/bootstrap\*.yaml`
 - `src/main/resources/db/`
 - `sql/`
 - `deploy/`
 - `infra/`
 - `secrets/`
- 不允许执行部署、推送、生产环境相关命令
- 如果需求边界不清楚，应停止并报告，不允许自行脑补需求

常用命令 (Maven)

- 编译: `mvn -q -DskipTests compile`
- 单测: `mvn test`
- 打包: `mvn -DskipTests package`

REVIEW.md

评审目标

name: reviewer

description: 只读评审代理，专门检查 OpenSpec 对齐、Spring Boot 分层问题和测试缺口

tools: [Read, Grep, Glob, LS]

model: sonnet

你是一个严格的只读评审代理。

你的职责：

- 对照 OpenSpec 工件检查实现是否一致

- 检查是否存在范围漂移
- 检查是否违反 Spring Boot 分层架构
- 检查是否把业务逻辑写进 Controller
- 检查是否缺少测试、事务说明、SQL 风险说明
- 输出具体问题，不要输出空洞表扬

禁止：

- 修改文件
- 提出与本次需求无关的大规模重构
- 在测试不足、范围不清楚时给出模糊通过结论

必须阅读：

- `REVIEW.md`
- `CLAUDE.md`
- `docs/architecture/implicit-contracts.md`
- 对应的 OpenSpec change 文件
- 本次改动涉及的源代码文件

OpenSpec注意事项：openspec/config.yaml

```
context: |
  历史隐性约定：
  - 某些前后端行为依赖 docs/architecture/implicit-contracts.md
  - 修改状态字段、接口返回结构、默认值语义时，必须先检查该文件
  - 如果需求触及这些约定，design.md 必须明确影响范围和兼容策略

rules:
  design:
    - 若涉及历史隐性约定，必须写清楚兼容处理方式
  tasks:
    - 若涉及历史隐性约定，必须包含验证步骤
```

知识库文件

架构总览：docs/architecture/index.md

架构总览

系统分层

- `controller`：接口层，负责请求接收、参数校验、响应封装
- `service`：业务编排层，负责核心业务流程
- `domain/entity`：领域对象或实体对象
- `repository/mapper`：数据访问层

- ``config``: 框架和中间件配置
- ``job`` / ``scheduler``: 定时任务

依赖方向

- controller -> service -> repository/mapper
- controller 不允许直接调用 mapper/repository
- repository/mapper 不允许反向依赖 service
- web 层对象不允许深入渗透到持久化层

高风险区域

以下区域修改时必须重点说明:

- 支付/订单
- 认证/鉴权
- 定时任务
- 数据迁移
- 批量更新/批量删除
- 核心 SQL

变更设计要求

如果 change 涉及高风险区域, design.md 中必须说明:

- 影响的类和模块
- 事务边界
- SQL / 索引 / 查询范围影响
- 回滚方案
- 测试策略

隐性约束: docs/architecture/implicit-contracts.md

隐性业务约定

广告投放状态字段 `status` 的隐性约定

- 背景: 历史上前后端口头约定, ``status = null`` 表示“未初始化”, ``status = 0`` 表示“已关闭”
- 代码现状: 后端代码中没有统一显式注释, 前端页面逻辑依赖这个区别
- 风险: 如果把 ``null`` 和 ``0`` 合并处理, 可能导致页面状态判断错误
- 处理要求:
 - 修改该字段相关逻辑前, 先确认前端依赖
 - 不要擅自把 `null` 归一化成默认值
 - 如确需调整, 必须在 OpenSpec design 中明确说明影响面

单元详情返回的时候需要有 `contentResponse`

- 背景: 历史上前后端口头约定, 后端会将前端传入的 `contentId` 转化成 `contentResponse`, 返回后用于前端展示
- 代码现状: 后端代码中没有统一显式注释, 前端页面逻辑依赖 `contentResponse` 进行展示
- 风险: 如果缺失 `contentResponse` 前端展示会报错

- 处理要求：
 - 修改该接口相关逻辑时，需要保留当前contentResponse逻辑
 - 如确需调整，必须在 OpenSpec design 中明确说明影响面

产品规则：docs/product/index.md

产品规则

核心原则

- 用户可见行为必须在 spec 中明确写出
- 除非 change 明确说明，否则优先保持向后兼容
- 不允许出现静默行为变化
- 错误码、提示文案、接口返回结构变更必须明确说明

验收标准

- 一个 change 只有在满足以下条件时才算通过：
 - 行为与 OpenSpec 工件一致
 - 必要测试通过
 - Review 没有未解决的严重问题
 - 关键风险已说明

测试规范：docs/standards/testing.md

测试规范

最低要求

- 行为变化：至少补 1 个相关测试
- Bug 修复：优先补 regression test
- Service 层逻辑修改：优先补单元测试
- Controller/API 改动：如适用，补集成测试
- 核心 SQL 改动：至少说明验证方法和边界数据

常用命令 (Maven)

- 单元测试：`mvn test`
- 编译检查：`mvn -q -DskipTests compile`
- 打包检查：`mvn -DskipTests package`

评审要求

提交 review 时必须说明：

- 跑了哪些测试
- 哪些部分没有测
- 为什么跳过这些检查仍然可以接受

数据库和SQL规范

数据库与 SQL 规范

基本规则

- 不允许无条件 UPDATE / DELETE
- 批量更新必须明确 where 条件和影响范围
- 分页查询必须明确排序条件
- 不允许在高频接口中引入明显的 N+1 查询
- 修改 Mapper / XML / SQL 时，必须关注索引命中情况

必须说明的内容

以下场景必须在 design.md 或 review 摘要中说明：

- 是否影响索引
- 是否会放大扫描范围
- 是否会引入锁竞争
- 是否需要数据修复或迁移
- 是否影响历史数据兼容性

高风险目录

- `\*\*/src/main/resources/mapper/`
- `\*\*/src/main/resources/db/`
- `sql/`

Claude 设置

权限设置：.claude/settings.json

```
{
  "defaultMode": "default",
  "permissions": {
    "allow": [
      "Bash(mvn test *)",
      "Bash(mvn -q -DskipTests compile *)",
      "Bash(mvn -DskipTests package *)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Skill(implement-from-openspec *)",
      "Skill(review-openspec-change *)",
      "Skill(prepare-review *)"
    ],
    "ask": [
      "Bash(*)",
      "Edit(*)",
    ]
  }
}
```



```

        "Write(*)"
    ],
    "deny": [
        "Bash(git push *)",
        "Bash(kubectl *)",
        "Bash(terraform *)",
        "Bash(helm *)",
        "Bash(rm -rf *)",
        "Read(secrets/**)",
        "Edit(secrets/**)",
        "Write(secrets/**)",
        "Edit(infra/**)",
        "Write(infra/**)",
        "Edit(deploy/**)",
        "Write(deploy/**)",
        "Edit(src/main/resources/application*)",
        "Write(src/main/resources/application*)",
        "Edit(src/main/resources/bootstrap*)",
        "Write(src/main/resources/bootstrap*)",
        "Edit(src/main/resources/db/**)",
        "Write(src/main/resources/db/**)",
        "Edit(sql/**)",
        "Write(sql/**)"
    ]
},
"hooks": {
    "PreToolUse": [
        {
            "matcher": "Edit|Write",
            "hooks": [
                {
                    "type": "command",
                    "command": "python3 .claude/hooks/guard_write.py",
                    "timeout": 10
                }
            ]
        }
    ],
    {
        "matcher": "Bash",
        "hooks": [
            {
                "type": "command",
                "command": "python3 .claude/hooks/ensure_change_context.py",
                "timeout": 10
            }
        ]
    }
]

```

```

    }
  ],
  "PostToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [
        {
          "type": "command",
          "command": "bash .claude/hooks/run_checks.sh",
          "timeout": 180
        }
      ]
    }
  ]
}

```

项目级权限设置：.claude/settings.local.json.example

```

{
  "permissions": {
    "allow": [
      "Bash(java -version)",
      "Bash(mvn -version)"
    ]
  }
}

```

Skills

变更摘要审计：.claude/skills/prepare-review/SKILL.md

```

---
name: prepare-review
description: 在 review 或提 PR 前，整理一份 Spring Boot 变更摘要
disable-model-invocation: true
---

# 生成评审摘要

## 输入
`$ARGUMENTS` = OpenSpec 的 change id

## 必读文件

```

- `openspec/changes/\$ARGUMENTS/proposal.md`
- `openspec/changes/\$ARGUMENTS/design.md`
- `openspec/changes/\$ARGUMENTS/tasks.md`
- 相关源码改动
- `docs/standards/testing.md`
- `docs/standards/database.md`

输出内容

- 本次变更的目标
- 影响到的类和模块
- 是否涉及接口、事务、SQL、配置
- 已运行测试
- 已知风险
- 仍未解决的问题

规则

- 不要写空洞表扬
- 保持简洁、事实化
- 优先标出风险和未完成项

Spring架构检查：.claude/skills/spring-architecture-review/SKILL.md

name: spring-architecture-review

description: 检查 Spring Boot 分层、依赖方向和业务逻辑放置是否合理

disable-model-invocation: true

Spring 架构审查

输入

`\$ARGUMENTS` = 本次改动涉及的文件、目录或 change id

检查项

1. Controller 是否写入了业务逻辑
2. Controller 是否直接调用 Mapper / Repository
3. Service 是否依赖了 Web 层对象
4. DTO/V0 是否被直接当作实体持久化
5. 是否存在明显的跨层耦合
6. 是否有可以归为 Domain/Service 的逻辑散落在其他层

输出格式

- 严重问题

- 警告问题
- 建议项

SQL风险检查：.claude/skills/sql-risk-review/SKILL.md

```
----
name: sql-risk-review
description: 检查 SQL、Mapper、批量更新、分页和索引相关风险
disable-model-invocation: true
----

# SQL 风险审查

## 输入
`$ARGUMENTS` = 本次改动涉及的 SQL、Mapper、XML、Repository 文件

## 检查项
1. 是否存在无条件更新/删除
2. 批量更新是否有清晰范围
3. 分页是否有稳定排序
4. 是否可能放大扫描范围
5. 是否存在明显的 N+1 风险
6. 是否缺少索引影响说明或验证方式

## 输出格式
- 严重问题
- 警告问题
- 建议项
```

子Agent

评审子Agent：.claude/agents/reviewer.md

```
----
name: reviewer
description: 只读评审代理，专门检查 OpenSpec 对齐、Spring Boot 分层问题和测试缺口
tools: [Read, Grep, Glob, LS]
model: sonnet
----
```

你是一个严格的只读评审代理。

你的职责：

- 对照 OpenSpec 工件检查实现是否一致

- 检查是否存在范围漂移
- 检查是否违反 Spring Boot 分层架构
- 检查是否把业务逻辑写进 Controller
- 检查是否缺少测试、事务说明、SQL 风险说明
- 输出具体问题，不要输出空洞表扬

禁止：

- 修改文件
- 提出与本次需求无关的大规模重构
- 在测试不足、范围不清楚时给出模糊通过结论

必须阅读：

- `REVIEW.md`
- `CLAUDE.md`
- 对应的 OpenSpec change 文件
- 本次改动涉及的源代码文件

Hooks

写入保护：.claude/hooks/guard_write.py

```
#!/usr/bin/env python3
import json
import sys
import fnmatch

data = json.load(sys.stdin)
tool_input = data.get("tool_input", {})
file_path = tool_input.get("file_path", "") or ""

blocked_patterns = [
    "*/src/main/resources/application*.yml",
    "*/src/main/resources/bootstrap*.yml",
    "*/src/main/resources/db/*",
    "*/sql/*",
    "*/deploy/*",
    "*/infra/*",
    "*/secrets/*"
]

for pattern in blocked_patterns:
    if fnmatch.fnmatch(file_path, pattern):
        print(json.dumps({
            "hookSpecificOutput": {
                "hookEventName": "PreToolUse",
```

```

        "permissionDecision": "deny",
        "permissionDecisionReason": f"禁止修改受保护路径: {pattern}"
    }
}))
sys.exit(0)

print(json.dumps({
    "hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": "allow",
        "permissionDecisionReason": "允许修改该路径"
    }
}))

```

上下文变更保护: .claude/hooks/ensure_change_context.py

```

#!/usr/bin/env python3
import json
import os
import sys

data = json.load(sys.stdin)
tool_input = data.get("tool_input", {})
cmd = tool_input.get("command", "") or ""

safe_prefixes = [
    "git status",
    "git diff",
    "mvn test",
    "mvn -q -DskipTests compile",
    "mvn -DskipTests package"
]

if any(cmd.startswith(p) for p in safe_prefixes):
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "allow",
            "permissionDecisionReason": "安全命令, 允许执行"
        }
    }))
    sys.exit(0)

change_dir = "openspec/changes"
has_change = os.path.isdir(change_dir) and any(

```

```

    os.path.isdir(os.path.join(change_dir, x))
    for x in os.listdir(change_dir)
)

if not has_change:
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "ask",
            "permissionDecisionReason": "当前未检测到 OpenSpec change, 请确认是
否继续"
        }
    }))
    sys.exit(0)

print(json.dumps({
    "hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": "allow",
        "permissionDecisionReason": "已检测到 OpenSpec change"
    }
}))

```

编译检查脚本: .claude/hooks/run_checks.sh

```

#!/bin/bash

# 1. 获取当前所有被修改、新增或删除的文件列表
CHANGED_FILES=$(git status --porcelain | awk '{print $2}')

# 2. 如果没有文件变动（极少见情况），直接退出
if [ -z "$CHANGED_FILES" ]; then
    exit 0
fi

# 3. 检查是否*只*修改了文档文件
# 过滤掉 .md, .txt, .json 等不需要跑测试的文件后缀
NON_DOC_CHANGES=$(echo "$CHANGED_FILES" | grep -vE '\.(md|txt|csv|json)$')

# 4. 判断逻辑
if [ -z "$NON_DOC_CHANGES" ]; then
    # 如果过滤后为空，说明这次改动的全是文档
    echo "[Hook 拦截] 仅检测到文档变动，跳过单元测试和语法检查。"
    exit 0
else

```

```
# 只要包含哪怕一个非文档文件（比如 .py 或 .java），就老老实实跑测试
echo "[Hook 触发] 检测到代码变动，开始执行全量检查..."
# 下面放你原本的测试命令，比如：
# pytest tests/

set -euo pipefail

echo "[hook] 开始执行编译检查..."
mvn -q -DskipTests compile

echo "[hook] 开始执行单元测试..."
mvn test

echo "[hook] 开始执行打包检查..."
mvn -DskipTests package
fi
```